Automatisering och integration av IT och OT system

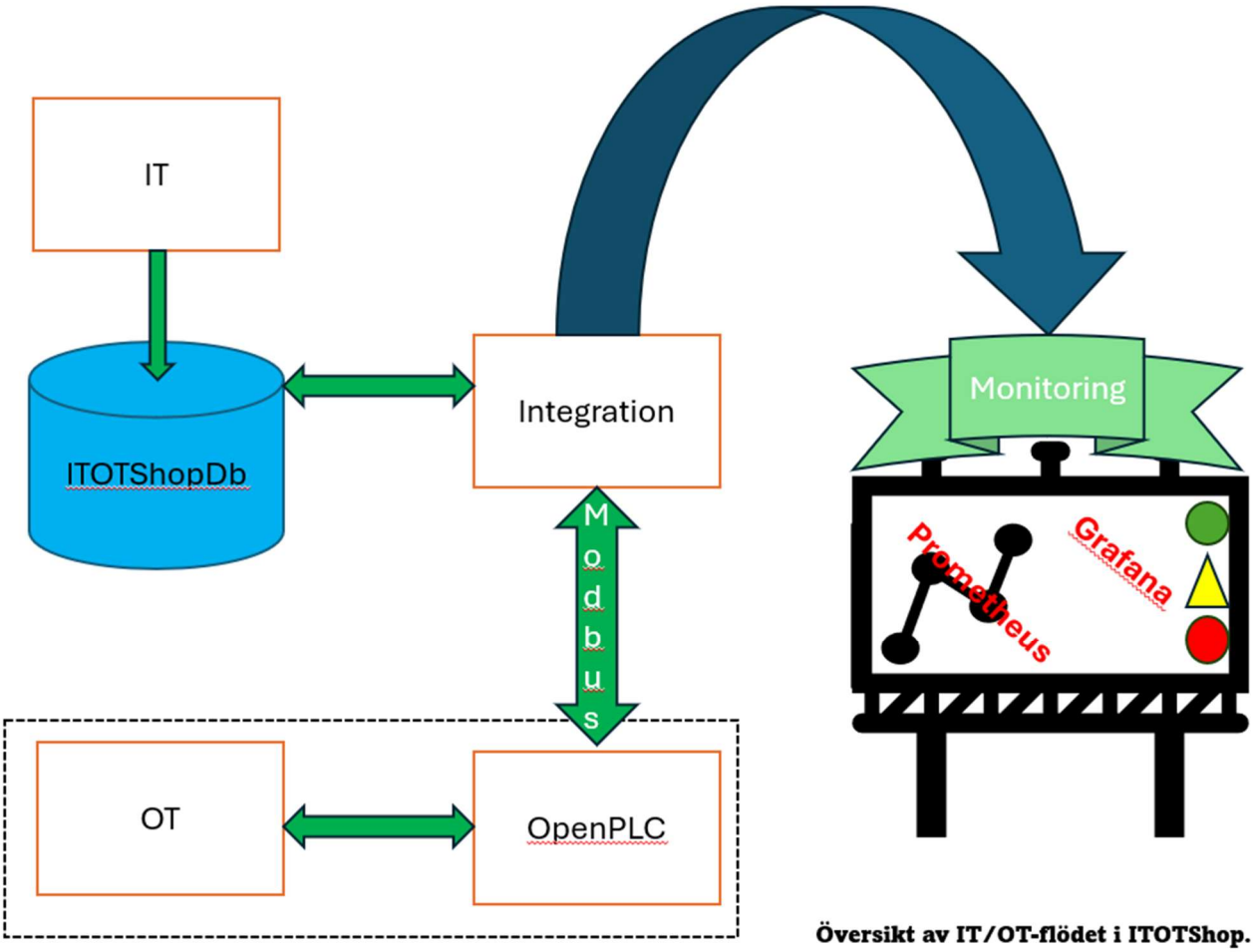
ITOTShop

Individuell inlämningsuppgift

Namn: **Siyament Ibrahimzade**

Kurs: **Automatisering och integration**

Datum: **2026-10-01**



Översikt av IT/OT-flödet i ITOTShop.

Innehållsförteckning

Innehållsförteckning 2
1. Inledning 3
1.1 Bakgrund och problemområde 3
1.2 Min lösning 3
2. Design och arkitektur 4
2.1 Projektens ansvar 4
2.2 Databas 6
2.3 Kommunikation mellan IT och OT 6
3. Implementering 7
3.1 Orderflödet 7
3.2 Hantering av avbrott 7
3.3 Felhantering i PLC 7
3.4 Monitoring med OpenTelemetry 9
4. Säkerhetsanalys och åtgärder 9
4.1 Separation mellan IT och OT 9
4.2 Autentisering 10
4.3 Roller 10
4.4 Validering 10
4.5 Risker 10
5. Installation och start av systemet 11
5.1 OpenPLC 11
5.2 Starta monitoring 12
5.3 Starta C#-projekten 13
5.4 Kontroll innan start 13
5.5 Starta en produktionskörning 14
5.6 Stänga ner systemet 14
6. Testning och resultat 15
6.1 Tester för ITOTShop.Console 15
6.2 Tester för ITOTShop.Integration 15
6.3 Tester för ITOTShop.OT 15
6.4 Manuella systemtester 15
6.5 Resultat 16
7. Slutsats och reflektion 17

1. Inledning

1.1 Bakgrund och problemområde

I ett tillverkningsföretag finns det behov av en tydlig gräns mellan IT-system och OT-system. På IT-sidan hanteras exempelvis användare, ordrar och databaser, medan OT-sidan ansvarar för själva produktionen genom PLC-system och annan industriell utrustning.

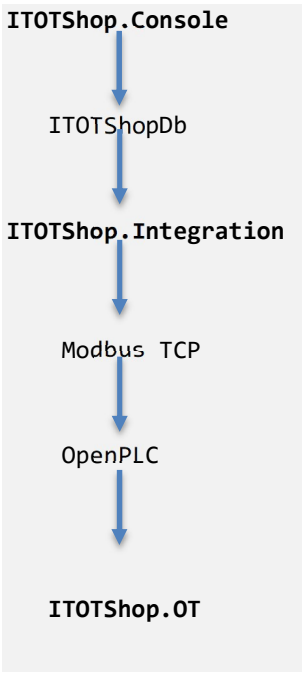
Om information mellan dessa system måste hanteras manuellt ökar risken för fel. En order kan exempelvis behöva registreras i ett system och sedan matas in manuellt i produktionssystemet. Det skapar extra arbete och kan leda till felaktiga värden, missade ordrar eller dålig spårbarhet.

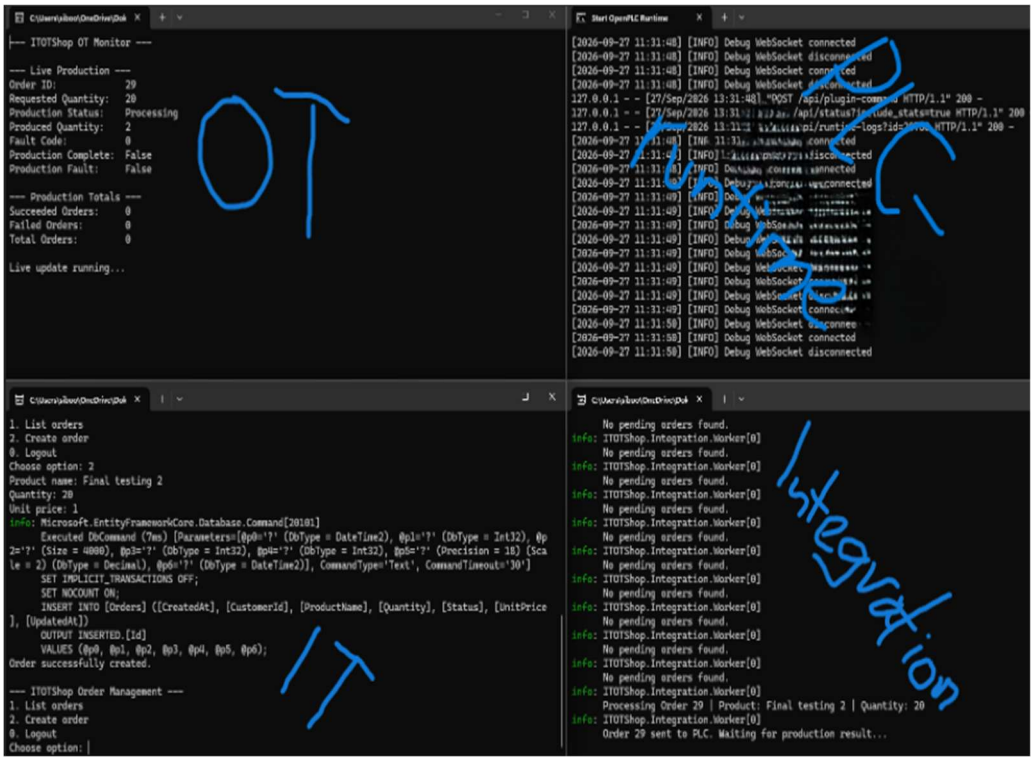
Syftet med mitt projekt har därför varit att bygga ett mindre system som visar hur en order automatiskt kan gå från ett IT-system till ett OT-system utan att användaren behöver mata in samma information flera gånger.

1.2 Min lösning

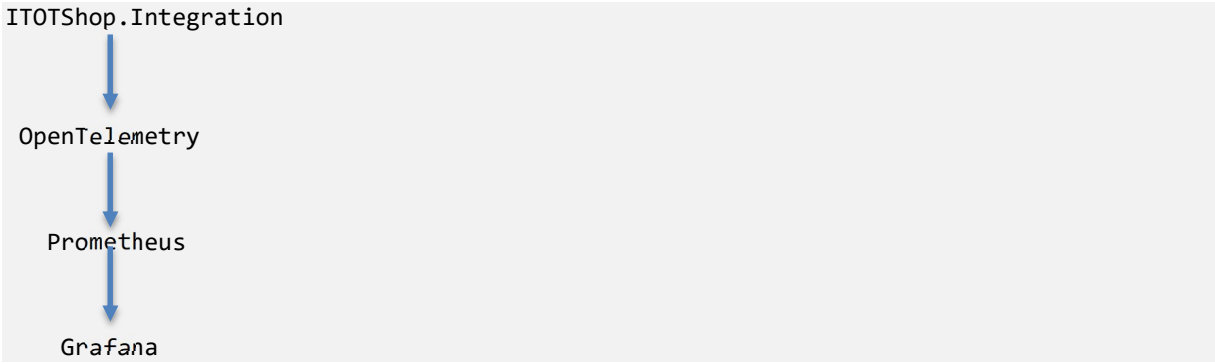
Jag har byggt lösningen i C# och .NET 10 och delat upp den i flera projekt med olika ansvar.

Huvudflödet är:





För övervakning används även:



En användare kan logga in i orderhanteringen och skapa en order. Ordern sparas i SQL Server-databasen ITOTShopDb. Integrationsprogrammet läser sedan ordern från databasen och skickar relevanta värden till OpenPLC via Modbus TCP. PLC:n simulerar därefter produktionen och integrationsprogrammet läser tillbaka resultatet. Produktionsdata och status kan samtidigt följas i Grafana.

2. Design och arkitektur

2.1 Projektens ansvar

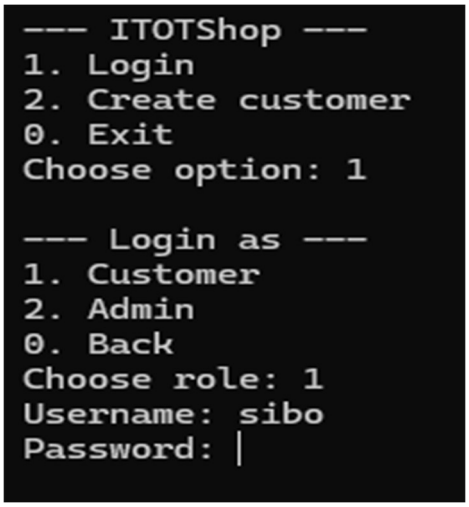
ITOTShop.Console

Detta projekt representerar IT-sidan och används för orderhantering.

Användaren kan bland annat:

- registrera en användare
- logga in
- skapa en order
- lista befintliga ordrar

Användarna och ordrarna lagras i ITOTShopDb.



ITOTShop.Integration

Integrationsprogrammet är länken mellan IT- och OT-sidan. Det ansvarar bland annat för att:

- läsa nästa order som ska behandlas
- ändra orderstatus
- skicka orderdata till PLC:n
- läsa PLC-status
- hantera kommunikationsfel
- rapportera metrics till Prometheus

Jag valde medvetet att låta integrationsprogrammet vara den komponent som kommunicerar med både databasen och OT-systemet. Orderhanteringen kommunicerar alltså inte direkt med PLC:n.

OpenPLC

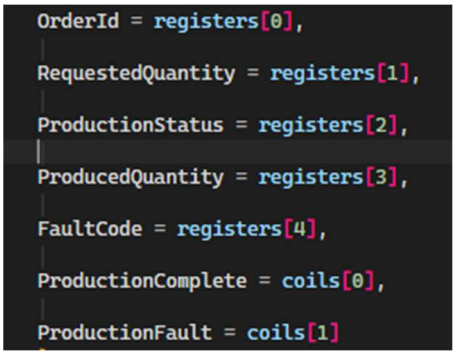
OpenPLC simulerar själva produktionssystemet. PLC-programmet tar emot information om vilken order som ska köras och hur många enheter som ska produceras. PLC:n håller bland annat reda på:

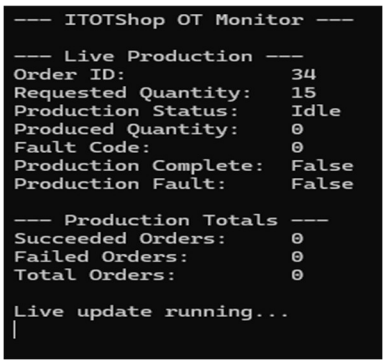
```
OrderId
RequestedQuantity
ProductionStatus
ProducedQuantity
FaultCode
TotalProducedQuantity
```

PLC:n använder även booleska signaler för exempelvis start av produktion och produktionsfel.

ITOTShop.OT

OT-projektet är ett enklare C#-program som används för att läsa och presentera information från OT-sidan. Det ger ytterligare insyn i PLC-systemets status och produktionsvärden.





ITOTShop.Tests

Testprojektet innehåller tester för Console-, Integration- och OT-delarna.

ITOTShop.Web

Jag skapade även ett webbprojekt under arbetets gång. Det ingår däremot inte i huvudflödet för den färdiga lösningen och saknar funktionalitet men är tänkt att utveckla utanför kursensram för att frigöra sig från konsol-användning samt träna på att ha systemen mer kompletta.

2.2 Databas

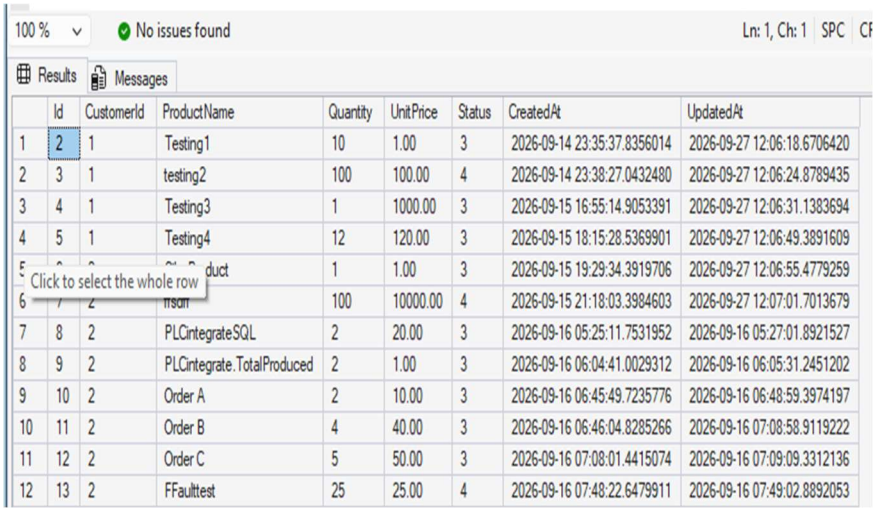
Databasen heter:

```
ITOTShopDb
```

SQL Server används tillsammans med Entity Framework Core. Databasen innehåller bland annat tabeller för användare och ordrar.

En order kan ha följande status:

```
Pending = 1
Processing = 2
Completed = 3
Failed = 4
```



	Id	CustomerId	ProductName	Quantity	UnitPrice	Status	CreatedAt	UpdatedAt
1	2	1	Testing1	10	1.00	3	2026-09-14 23:35:37.8356014	2026-09-27 12:06:18.6706420
2	3	1	testing2	100	100.00	4	2026-09-14 23:38:27.0432480	2026-09-27 12:06:24.8789435
3	4	1	Testing3	1	1000.00	3	2026-09-15 16:55:14.9053391	2026-09-27 12:06:31.1383694
4	5	1	Testing4	12	120.00	3	2026-09-15 18:15:28.5369901	2026-09-27 12:06:49.3891609
5			duct	1	1.00	3	2026-09-15 19:29:34.3919706	2026-09-27 12:06:55.4779259
6				100	10000.00	4	2026-09-15 21:18:03.3984603	2026-09-27 12:07:01.7013679
7	8	2	PLCintegrateSQL	2	20.00	3	2026-09-16 05:25:11.7531952	2026-09-16 05:27:01.8921527
8	9	2	PLCintegrate.TotalProduced	2	1.00	3	2026-09-16 06:04:41.0029312	2026-09-16 06:05:31.2451202
9	10	2	Order A	2	10.00	3	2026-09-16 06:45:49.7235776	2026-09-16 06:48:59.3974197
10	11	2	Order B	4	40.00	3	2026-09-16 06:46:04.8285266	2026-09-16 07:08:58.9119222
11	12	2	Order C	5	50.00	3	2026-09-16 07:08:01.4415074	2026-09-16 07:09:09.3312136
12	13	2	FFaulttest	25	25.00	4	2026-09-16 07:48:22.6479911	2026-09-16 07:49:02.8892053

När en ny order skapas får den status Pending. När integrationsprogrammet börjar behandla ordern ändras status till Processing. När PLC:n är klar ändras den sedan till Completed eller Failed beroende på händelseförloppet.

2.3 Kommunikation mellan IT och OT

För kommunikationen mellan integrationsprogrammet och PLC:n valde jag Modbus TCP med NModbus.

OpenPLC lyssnar på:

```
127.0.0.1:5020
```

Integrationsprogrammet skriver bland annat ordernummer och antal enheter till PLC-register. Ett exempel på register som används är:

```
OrderId             -> register 0
RequestedQuantity   -> register 1
ProductionStatus    -> register 2
ProducedQuantity    -> register 3
FaultCode           -> register 4
```

Startsignalen skickas separat som en Modbus coil. Jag märkte under testerna att en mycket kort startsignal kunde missas av PLC:n. Därför håller integrationsprogrammet startsignalen aktiv en kort stund innan den återställs. Efter den ändringen blev orderflödet stabilare.

#	Name	Class	Type	Location	Initial Value	Documentation
0	OrderId	Local	UINT	%QW0	0	Order-ID from integration program
1	RequestedQuantity	Local	UINT	%QW1	0	Antal produkter som ska tillverkas
2	ProductionStatus	Local	UINT	%QW2	0	0=Idle, 1= Processing, 2=Completed, 3=Fault
3	ProducedQuantity	Local	UINT	%QW3	0	Antal färdig producerade produkter
4	FaultCode	Local	UINT	%QW4	0	Felkod från produktionsprocessen
5	StartProduction	Local	BOOL	%QX0.0	FALSE	Startsignal från integrationsprogrammet
6	ResetProduction	Local	BOOL	%QX0.1	FALSE	Återställer produktionsprocessen
7	ProductionComplete	Local	BOOL	%QX0.2	FALSE	Visar att ordern är färdig
8	ProductionFault	Local	BOOL	%QX0.3	FALSE	Visar att ett produktionsfel har uppstått
9	TotalProducedQuantity	Local	UINT	%QW5	0	Totalt antal producerade enheter
10	ProductionTimer	Local	TON			TTimer som simulerar produktionstiden
11	ResetTimer	Local	BOOL		FALSE	Återställer timern mellan varje produkt

3. Implementering

3.1 Orderflödet

När en användare skapar en order i ITOTShop.Console sparas ordern först i databasen med status Pending. Integrationsprogrammet söker kontinuerligt efter en order som behöver behandlas.

Flödet ser förenklat ut så här:

```
Order skapas
    |
    v
Pending
    |
    v
Integration hämtar order
    |
    v
Processing
    |
    v
Order skickas till PLC
    |
    v
Produktion
    |
    +------> Completed
    |
    +------> Failed
```

När PLC:n är klar uppdateras statusen i databasen.

3.2 Worker – integrationsprogrammets kärna

Klassen Worker är själva kärnan i ITOTShop.Integration. Program.cs ansvarar främst för att konfigurera och starta applikationen, medan Worker utför det löpande integrationsarbetet.

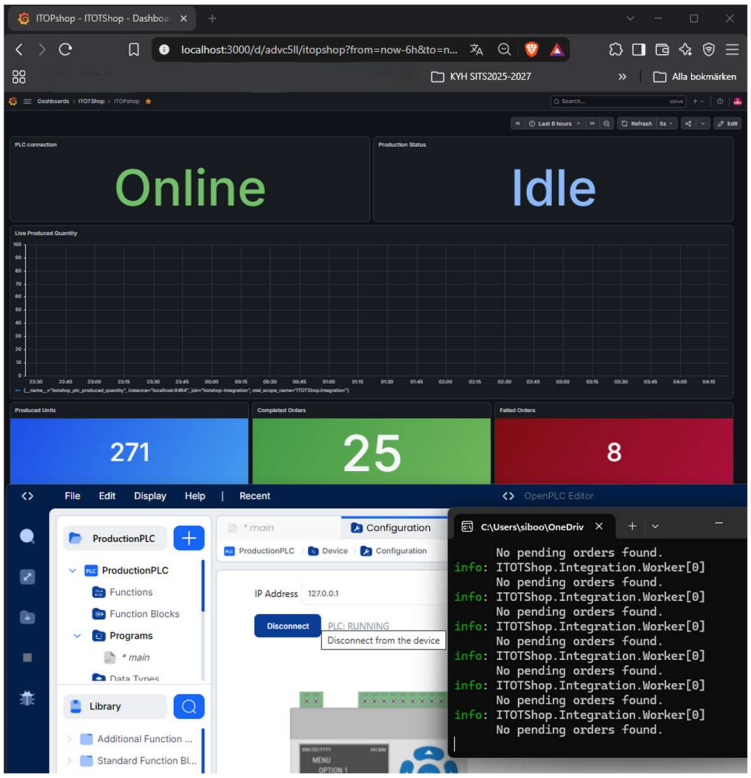
Worker ärver från BackgroundService och kör kontinuerligt så länge integrationsprogrammet är aktivt. Den skapar nya scopes för att kunna använda databastjänster, hämtar nästa order som ska behandlas och använder ModbusService för kommunikationen med OpenPLC.

När en order hittas ansvarar Worker för hela processen från databas till PLC och tillbaka igen. Den skickar orderdata till OpenPLC, ändrar orderstatus till Processing, läser produktionsstatus och väntar tills PLC rapporterar Completed eller ett riktigt produktionsfel.

Klassen hanterar även kommunikationsproblem. Om PLC tillfälligt inte går att nå markeras inte ordern direkt som Failed. Integrationsprogrammet väntar i stället och försöker igen tills kommunikationen återställs.

När produktionen är klar uppdateras orderstatus i databasen, statistiken uppdateras och PLC återställs till Idle innan nästa order behandlas.

Worker uppdaterar dessutom PLC-status och produktionsvärden även när det inte finns någon aktiv order. Det gör att Grafana fortfarande kan visa korrekt Online/Offline och aktuell produktionsstatus när systemet står i Idle.



3.3 Felhantering i PLC

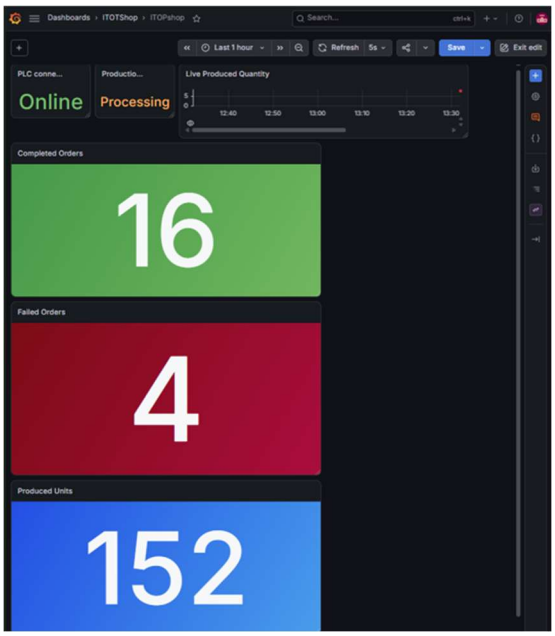
PLC-programmet validerar orderinformationen innan produktionen startar. Exempel på fel som hanteras är:

```
OrderId = 0
RequestedQuantity = 0
RequestedQuantity > 20
```

Dessa ger olika felkoder. Det gör att felaktiga värden inte direkt kan starta en normal produktionskörning.

```
28    ELSIF RequestedQuantity > 20 THEN
29        (* Simulerat produktionsfel - för stor order *)
30        ProductionStatus := 3;
31        FaultCode := 3;
32        ProductionFault := TRUE;
33
```

3.4 Monitoring med OpenTelemetry



Jag har implementerat OpenTelemetry i ITOTShop.Integration.

Integrationsprogrammet exponerar metrics via:

```
http://localhost:9464/metrics
```

Prometheus samlar in dessa värden och

Grafana använder sedan Prometheus som datakälla. Exempel på metrics är:

```
itotshop_orders_completed
itotshop_orders_failed
itotshop_units_produced
itotshop_plc_connected
itotshop_plc_produced_quantity
itotshop_plc_production_status
```

Prometheus och Grafana körs med Docker Compose.

Prometheus nås på:

```
http://localhost:9090
```

Grafana nås på:

```
http://localhost:3000
```

Grafana-dashboarden visar:

- PLC Connection
- Production Status
- Live Produced Quantity
- Produced Units
- Completed Orders
- Failed Orders

Under en normal produktionskörning kan Production Status följas som:

```
Idle -> Processing -> Completed -> Idle
```

Jag har även sett till att dashboarden inte fortsätter visa PLC:n som Online om integrationsprogrammet eller PLC:n har stängts av. Eftersom Prometheus och Grafana uppdaterar värden med ett visst intervall tar det några sekunder innan ändringen visas.

4. Säkerhetsanalys och åtgärder

4.1 Separation mellan IT och OT

En av de viktigaste säkerhetsprinciperna i lösningen är att orderhanteringen inte får kommunicera direkt med PLC:n.

Kommunikationen går i stället enligt:

```
Console -> Database -> Integration -> PLC
```

ITOTShop.Integration fungerar därmed som en tydlig integrationspunkt mellan IT- och OT-miljön. Det minskar antalet komponenter som behöver ha direkt åtkomst till OT-systemet.

I en verklig produktionsmiljö hade denna gräns behövt förstärkas ytterligare med exempelvis nätverkssegmentering, brandväggar och striktare autentisering mellan systemen.

4.2 Autentisering

Orderhanteringen har ett eget användarsystem. En användare måste registrera sig och logga in innan orderfunktionerna används.

Lösenorden sparas inte i klartext utan lagras som hashade lösenord. Det minskar risken för att användarnas faktiska lösenord exponeras om databasen skulle läsas av en obehörig.

4.3 Roller

Systemet använder rollerna:

```
Customer
Admin
```

Det ger möjlighet att separera funktioner beroende på vilken typ av användare som är inloggad. Det är viktigt eftersom alla användare inte ska ha samma behörighet i ett system som påverkar orderhantering och produktion.

4.4 Validering

Data valideras på flera nivåer. På PLC-sidan kontrolleras exempelvis att ordernummer och antal är rimliga innan produktionen startar. Detta är viktigt eftersom integrationssystem inte bör lita blint på data som kommer från ett annat system.

4.5 Risker

Det finns fortfarande risker i den lösning jag har byggt. Modbus TCP har exempelvis ingen egen modern autentisering eller kryptering. I projektet körs systemen lokalt, men i en riktig miljö hade jag inte velat exponera Modbus-kommunikationen direkt mot ett vanligt IT-nätverk.

Andra risker är exempelvis:

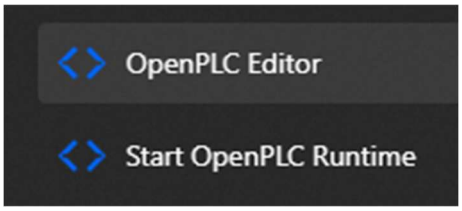
- obehörig åtkomst till databasen
- manipulation av orderdata
- förlorad kommunikation mellan Integration och PLC
- felaktiga produktionsvärden
- driftstopp i integrationsprogrammet

Min lösning minskar vissa av riskerna genom separation mellan komponenterna, autentisering, validering och felhantering, men den ska fortfarande ses som en simulerad lösning och inte som ett färdigt industriellt säkerhetssystem.

5. Installation och start av systemet

För att kunna köra hela lösningen behövs följande program och verktyg:

- Visual Studio med stöd för .NET 10
- SQL Server
- Docker Desktop
- OpenPLC Editor
- OpenPLC Runtime



OpenPLC består av två delar i min lösning. **OpenPLC Editor** används för att öppna, bygga och ladda PLC-programmet, medan **OpenPLC Runtime** kör själva PLC-programmet. Editorn ansluter till Runtime för att överföra och starta PLC-programmet.

5.1 OpenPLC

Installera först:

OpenPLC Editor

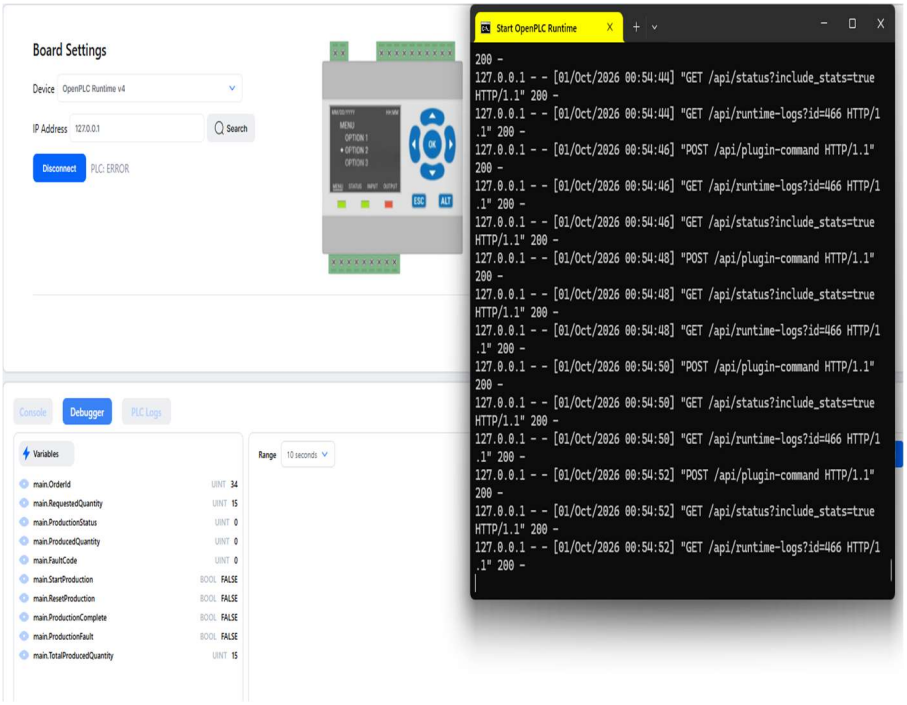
OpenPLC Runtime

OpenPLC Runtime behöver vara igång innan produktionsflödet kan köras.

I mitt projekt används Runtime som det simulerade PLC-systemet och Modbus TCP används för kommunikationen mellan ITOTShop.Integration och OpenPLC.

Modbus-kommunikationen använder:

127.0.0.1:5020



Starta därefter OpenPLC Editor och anslut Editorn till OpenPLC Runtime.

Öppna PLC-projektet, bygg projektet och ladda upp det till Runtime.

När PLC-programmet är uppladdat och startat är OT-delen redo att ta emot orderinformation från integrationsprogrammet.

5.2 Starta monitoring

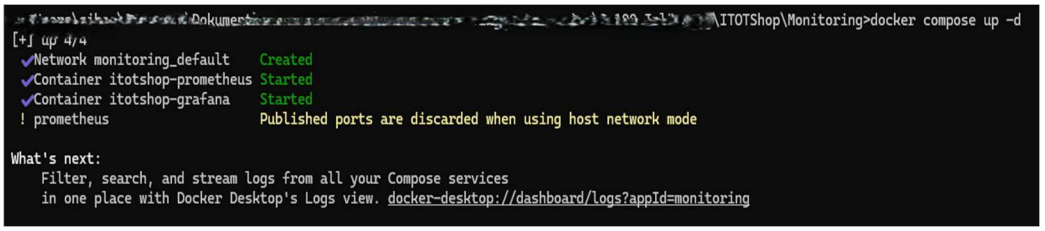
Prometheus och Grafana körs i Docker.

Öppna en terminal i:

Monitoring

och kör:

docker compose up -d

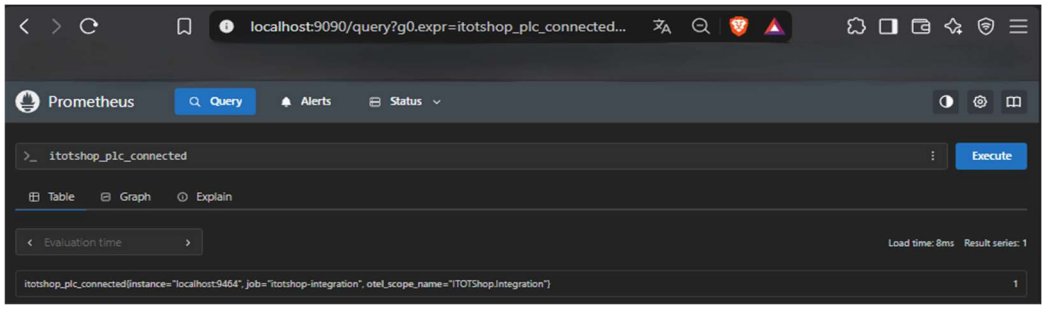


Det startar:

Prometheus & Grafana

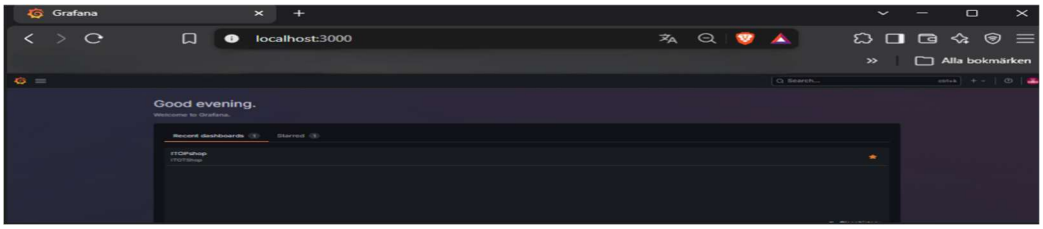
Prometheus kan därefter nås på:

http://localhost:9090



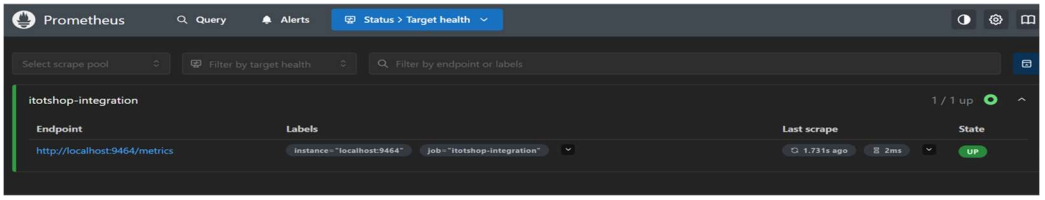
Grafana på:

http://localhost:3000



Integrationsprogrammet exponerar sina metrics på:

http://localhost:9464/metrics



5.3 Starta C#-projekten

I Visual Studio används **Multiple startup projects**.

Följande projekt ska startas samtidigt:

ITOTShop.Console

ITOTShop.Integration

ITOTShop.OT

ITOTShop.Web behöver inte startas eftersom det inte ingår i huvudflödet för inlämningen.

När projekten startas får man tre separata program:

- ITOTShop.Console för orderhantering och inloggning
- ITOTShop.Integration för kommunikationen mellan IT och OT
- ITOTShop.OT för övervakning av OT-data

5.4 Kontroll innan start

Innan en order skapas bör följande vara igång:

OpenPLC Runtime

PLC-programmet i OpenPLC

Prometheus

Grafana

ITOTShop.Console

ITOTShop.Integration

ITOTShop.OT

I Grafana ska PLC-status då visa:

PLC Connection -> Online

Production Status -> Idle

Det visar att integrationsprogrammet har kontakt med PLC och att systemet är redo för en ny order.

5.5 Starta en produktionskörning

När alla delar är igång kan produktionsflödet testas.

1. Logga in i ITOTShop.Console.
2. Skapa en ny order.
3. Ordern sparas i ITOTShopDb med status Pending.
4. ITOTShop.Integration hämtar ordern.
5. Orderstatus ändras till Processing.
6. OrderId och Quantity skickas till OpenPLC via Modbus TCP.
7. PLC startar den simulerade produktionen.
8. Producerad kvantitet skickas tillbaka.
9. Ordern ändras till Completed eller Failed.
10. Grafana uppdaterar produktionsstatus och statistik.

Vid en lyckad körning kan statusen i Grafana följas som:

Idle -> Processing -> Completed -> Idle

Samtidigt uppdateras bland annat:

Produced Units

Completed Orders

Live Produced Quantity

5.6 Stänga ner systemet

När testet är klart kan C#-projekten och OpenPLC stoppas.

Prometheus och Grafana kan stängas ner från Monitoring med:

docker compose down

Docker-volymerna behöver normalt inte tas bort eftersom de används för att behålla Grafanas inställningar och Prometheus historiska data.

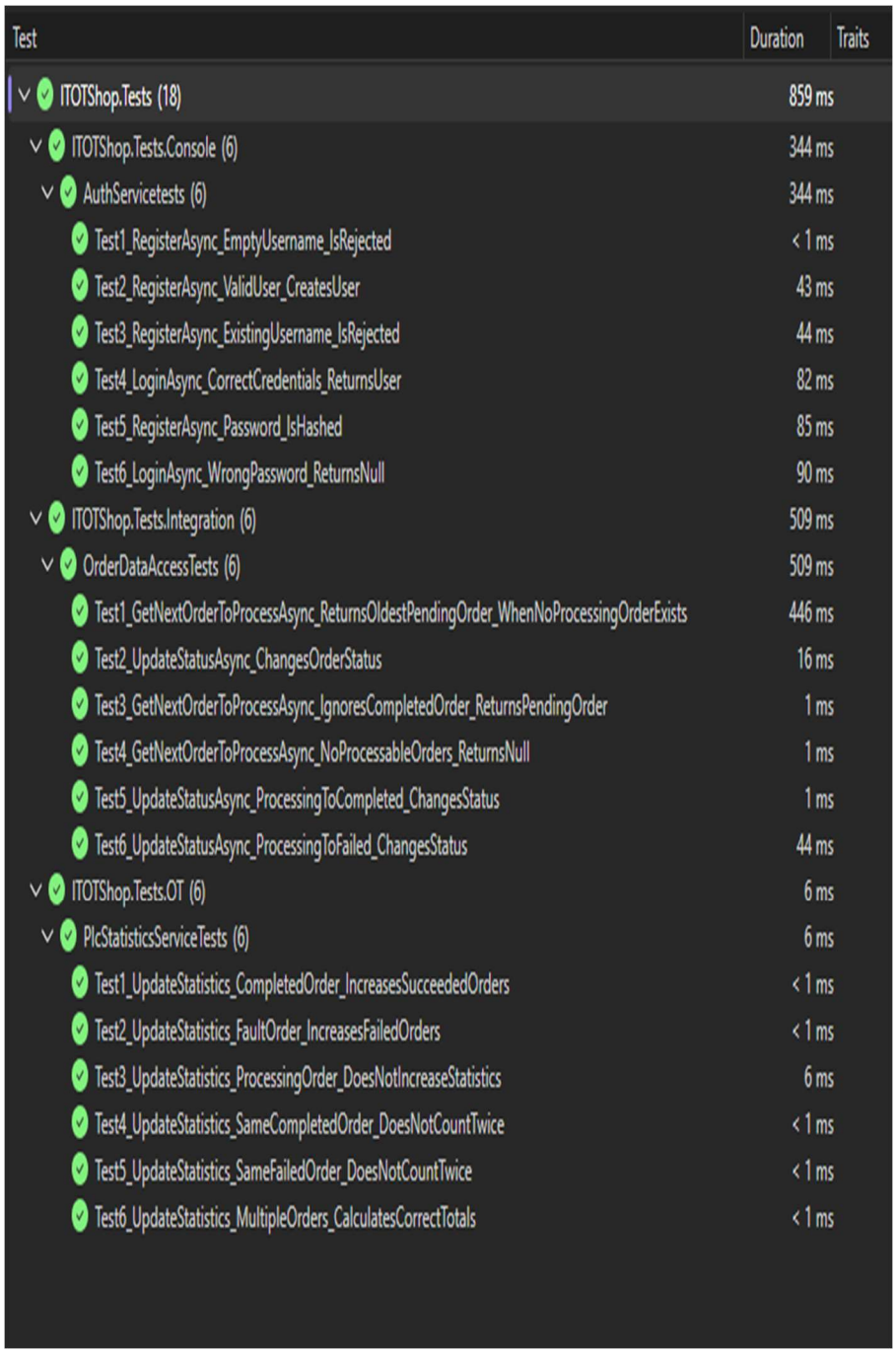
6. Testning och resultat

Jag har både testat systemet manuellt och byggt automatiserade tester med xUnit.

Totalt finns:

```
18 tester
```

Samtliga tester går igenom.



6.1 Tester för ITOTShop.Console

Det finns 6 tester för autentiseringsdelen. De kontrollerar bland annat registrering och inloggning samt att användarinformation hanteras korrekt. Detta är viktigt eftersom inloggningen är en del av säkerhetslösningen på IT-sidan.

6.2 Tester för ITOTShop.Integration

Det finns 6 tester för OrderDataAccess. De kontrollerar bland annat:

- hämtning av order
- uppdatering av status
- att färdigbehandlade ordrar inte väljs som nya ordrar
- att null returneras när det inte finns någon behandlingsbar order
- övergång från Processing till Completed
- övergång från Processing till Failed

6.3 Tester för ITOTShop.OT

Det finns 6 tester för PlcStatisticsService. De testar logiken för hantering och presentation av produktionsdata från PLC-systemet.

6.4 Manuella systemtester

Utöver de automatiserade testerna har jag testat hela kedjan. Ett normalt test har varit:

1. Starta OpenPLC
2. Starta Prometheus och Grafana
3. Starta Console, Integration och OT
4. Logga in i Console
5. Skapa en order

6. Kontrollera att ordern får status Pending
7. Kontrollera att Integration hämtar ordern
8. Kontrollera Processing i Grafana
9. Kontrollera att PLC producerar rätt antal
10. Kontrollera Completed
11. Kontrollera att systemet går tillbaka till Idle

Jag har även testat fel- och avbrottsscenarier. Exempelvis har jag stängt av PLC:n medan Integration varit igång.

Grafana visar då: PLC Connection -> Offline och Production Status -> No data

Jag har även stoppat integrationsprogrammet medan PLC:n varit igång. Efter några sekunder visar Grafana att anslutningen är Offline.

När både PLC och Integration körs igen visas: PLC Connection -> Online och Production Status -> Idle

Detta visar att statusen i dashboarden följer systemets faktiska läge och inte bara visar ett gammalt mätvärde.

6.5 Resultat

Vid sluttestet fungerade hela orderflödet.

En testorder gick från: Pending → Processing → Completed

samtidigt som Grafana visade: Idle → Processing → Completed → Idle

Produced Units och Completed Orders ökade korrekt och Failed Orders förändrades inte när ordern lyckades.

7. Slutsats och reflektion

Projektet har varit både givande och stundtals utmanande. Från början trodde jag att systemen och kommunikationen mellan dem skulle vara enklare än vad det faktiskt blev. Under arbetets gång blev det tydligt att en IT/OT-integration innehåller många fler delar än att bara skicka data från ett system till ett annat.

Vissa fel har också varit svåra att upptäcka eftersom de inte alltid har legat i själva koden. Ett tydligt exempel var startsignalen till OpenPLC. Integrationsprogrammet skickade rätt värden, men PLC:n hann inte alltid reagera när signalen låg aktiv för kort tid. Det gjorde felsökningen svårare eftersom båda systemen såg ut att fungera var för sig, samtidigt som hela flödet ändå kunde stanna.

Ett annat problem var ordrar som redan hade fått status Processing. Om integrationsprogrammet stoppades behövde systemet kunna hantera ordern när det startade igen. Det ledde till att jag ändrade logiken så att pågående ordrar prioriteras före nya Pending-ordrar.

Monitoring-delen blev också mer omfattande än jag först hade tänkt. Framför allt kommunikationen mellan Integration, Prometheus och Grafana krävde en hel del felsökning kring Docker, nätverk och hur metrics skulle exponeras. När det väl fungerade blev resultatet däremot tydligt och användbart eftersom jag kunde följa produktionen och se om PLC:n verkligen var online eller offline.

Det jag framför allt tar med mig från projektet är att integration mellan IT och OT kräver att man tänker på hela kedjan. Det räcker inte att systemen fungerar var för sig. Man måste även ta hänsyn till timing, kommunikationsfel, felaktiga värden och vad som händer om någon del av systemet startas om eller försvinner.

Ur säkerhetssynpunkt finns det fortfarande mycket som skulle behöva göras i ett verkligt industriellt system. Framför allt hade kommunikationen mellan IT och OT behövt skyddas mer och nätverken hade behövt segmenteras tydligare.

Som skolprojekt tycker jag däremot att lösningen visar hela kedjan från IT till OT:

```
Användare
   |
Orderhantering
   |
Databas
   |
Integration
   |
Modbus
   |
  PLC
   |
Produktion
   |
Monitoring
```